Summarize

Chapter 5. Adapters

In the preceding chapters, we saw how the sidecar pattern can extend and augment existing application containers. We also saw how ambassadors can alter and broker how an application container communicates with the external world. This chapter describes the final single-node pattern: the *adapter* pattern. In the adapter pattern, the *adapter container* is used to modify the interface of the *application container* so that it conforms to some predefined interface that is expected of all applications. For example, an adapter might ensure that an application implements a consistent monitoring interface. Or it might ensure that log files are always written to `stdout` or any number of other conventions.

Real-world application development is a heterogeneous, hybrid exercise. Some parts of your application might be written from scratch by your team, some supplied by vendors, and some might consist entirely of off-the-shelf open source or proprietary software that you consume as precompiled binary. The net effect of this heterogeneity is that any real-world application you deploy will have been written in a variety of languages, with a variety of conventions for logging, monitoring, and other common services.

Yet, to effectively monitor and operate your application, you need common interfaces. When each application provides metrics using a different format and interface, it is very difficult to collect all of those metrics in a single place for visualization and alerting. This is where the adapter pattern is relevant. Like other single-node patterns, the adapter pattern is made up of modular containers. Different application containers can present many different monitoring interfaces while the adapter container adapts this heterogeneity to present a consistent interface. This enables you to deploy a single tool that expects this single interface. Figure 5-1 illustrates this general pattern.

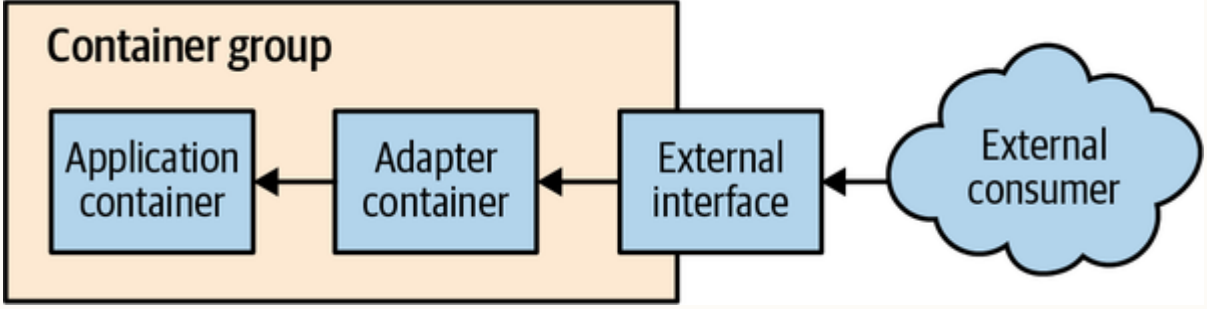


Figure 5-1. The generic adapter pattern

The remainder of this chapter gives several different applications of the adapter pattern.

Monitoring

When monitoring your software, you want a single solution that can automatically discover and monitor any application that is deployed into your environment. To make this feasible, every application has to implement the same monitoring interface. There are numerous examples of standardized monitoring interfaces, such as `syslog`, Event Tracing for Windows (ETW), JMX for Java applications, and many, many other protocols and interfaces. However, each of these is unique in both protocol for communication as well as the style of communication (push versus pull).

Sadly, applications in your distributed system are likely to span the gamut from code that you have written yourself to off-the-shelf open source components. As a result, you will find yourself with a wide range of different monitoring interfaces that you need to integrate into a single well-understood system.

Fortunately, most monitoring solutions understand that they need to be widely applicable, and thus they have implemented a variety of plug-ins that can adapt one monitoring format to a common interface. Given this set of tools, how can we deploy and manage our applications in an agile and stable manner? Fortunately, the adapter pattern can provide us with the answers. Applying the adapter pattern to monitoring, we see that the application container is simply the application that we want to monitor. The adapter container contains the tools for transforming the monitoring interface exposed by the application container into the interface expected by the general-purpose monitoring system.

Decoupling the system in this fashion makes for a more comprehensible, maintainable system. Rolling out new versions of the application doesn't require a rollout of the monitoring adapter. Additionally, the monitoring container can be reused with multiple different application containers. The monitoring container may even have been supplied by the monitoring system maintainers independent of the application developers. Finally,

deploying the monitoring adapter as a separate container ensures that each container gets its own dedicated resources in terms of both CPU and memory. This ensures that a misbehaving monitoring adapter cannot cause problems with a user-facing service.

Hands On: Using Prometheus for Monitoring

As an example, consider monitoring your containers via the Prometheus open source project. Prometheus is a monitoring aggregator, which collects metrics and aggregates them into a single time-series database. On top of this database, Prometheus provides visualization and query language for introspecting the collected metrics. To collect metrics from a variety of different systems, Prometheus expects every container to expose a specific `metrics` API. This enables Prometheus to monitor a wide variety of different programs through a single interface.

However, many popular programs, such as the Redis key-value store, do not export metrics in a format that is compatible with Prometheus. Consequently, the adapter pattern is quite useful for taking an existing service like Redis and adapting it to the Prometheus metrics-collection interface.

Consider a simple Kubernetes pod definition for a Redis server:

```
apiVersion: v1
kind: Pod
metadata:
  name: adapter-example
  namespace: default
spec:
  containers:
  - image: redis
    name: redis
```

At this point, this container is not capable of being monitored by Prometheus because it does not export the right interface. However, if we simply add an adapter container (in this case, an open source Prometheus exporter), we can modify this pod to export the correct interface and thus adapt it to fit Prometheus's expectations:

```
apiVersion: v1
kind: Pod
metadata:
  name: adapter-example
  namespace: default
```

```
spec:
  containers:
  - image: redis
    name: redis
  # Provide an adapter that implements the Prometheus interface
  - image: oliver006/redis_exporter
    name: adapter
```

This example illustrates not only the value of the adapter pattern for ensuring a consistent interface, but also the value of container patterns in general for modular container reuse. In this case, the example shown combines an existing Redis container with an existing Prometheus adapter. The net effect is a monitorable Redis server, with little work on our part to deploy it. In the absence of the adapter pattern, the same deployment would have required significantly more custom work and would have resulted in a much less operable solution, since any updates to either Redis or the adapter would have required work to apply the update.

Logging

Much like monitoring, there is a wide variety of heterogeneity in how systems log data to an output stream. Systems might divide their logs into different levels (such as debug, info, warning, and error) with each level going into a different file. Some might simply log to `stdout` and `stderr`. This is especially problematic in the world of containerized applications, where there is a general expectation that your containers will log to `stdout`, because that is what is available via commands like `docker logs` or `kubectl logs`.

Adding further complexity, the information logged generally has structured information (e.g., the date/time of the log), but this information varies widely between different logging libraries (e.g., Java's built-in logging versus the `glog` package for Go).

Of course, when you are storing and querying the logs for your distributed system, you don't really care about these differences in logging format. You want to ensure that despite different structures for the data, every log ends up with the appropriate timestamp.

Fortunately, as with monitoring, the adapter pattern can help provide a modular, re-usable design for both of these situations. While the application container may log to a file, the adapter container can redirect that file to `stdout`. Different application containers can log information in different formats, but the adapter container can transform that

data into a single structured representation that can be consumed by your log aggregator. Again, the adapter is taking a heterogeneous world of applications and creating a homogenous world of common interfaces.

NOTE

One question that often comes up when considering adapter patterns is: why not simply modify the application container itself? If you are the developer responsible for the application container, then this might actually be a good solution. Adapting your code or your container to implement a consistent interface can work well. However, in many cases we are reusing a container produced by another party. In such cases, deriving a slightly modified image that we have to maintain (patch, rebase, etc.) is significantly more expensive than developing an adapter container that can run alongside the other party's image. Additionally, decoupling the adapter into its own container allows for the possibility of sharing and reuse, which isn't possible when you modify the application container.

Hands On: Normalizing Different Logging Formats with fluentd

One common task for an adapter is to normalize log metrics into a standard set of events. Many different applications have different output formats, but you can use a standard logging tool deployed as an adapter to normalize them all to a consistent format. In this example, we will use the `fluentd` monitoring agent as well as some community-supported plug-ins to obtain logs from a variety of different sources.

`fluentd` is one of the more popular open source logging agents available. One of its major features is a rich set of community-supported plug-ins that enable a great deal of flexibility in monitoring a variety of applications.

The first application that we will monitor is Redis. Redis is a popular key-value store; one of the commands it offers is the `SLOWLOG` command. This command lists recent queries that exceeded a particular time interval. Such information is quite useful in debugging your application's performance. Unfortunately, `SLOWLOG` is only available as a command on the Redis server, which means that it is difficult to use retrospectively if a problem happens when someone isn't available to debug the server. To fix this limitation, we can use `fluentd` and the adapter pattern to add slow-query logging to Redis.

To do this, we use the adapter pattern with a `redis` container as the main application container, and the `fluentd` container as our adapter container. In this case, we will also use the `fluent-plugin-redis-slowlog` `fluentd` plug-in to listen to the slow queries. We can configure this plug-in by using the following snippet:

```
<source>
  type redis_slowlog
  host localhost
  port 6379
  tag redis.slowlog
</source>
```

Because we are using an adapter and the containers both share a network namespace, configuring the logging simply uses `localhost` and the default Redis port (6379). Given this application of the adapter pattern, logging will always be available whenever we want to debug slow Redis queries.

A similar exercise can be done to monitor logs from the Apache Storm system. Again, Storm provides data via a RESTful API, which is useful but has limitations if we are not currently monitoring the system when a problem occurs. Like Redis, we can use a `fluentd` adapter to transform the Storm process into a time series of queryable logs. To do this, we deploy a `fluentd` adapter with the `fluent-plugin-storm` plug-in enabled. We can configure this plug-in with a `fluentd` config pointed at localhost (because again, we are running as a container group with a shared localhost); the config for the plug-in looks like:

```
<source>
  type storm
  tag storm
  url http://localhost:8080
  window 600
  sys 0
</source>
```

This adapter builds a bridge between Storm and the time series of logs.

Adding a Health Monitor

One last example of applying the adapter pattern is derived from monitoring the health of an application container. Consider the task of monitoring the health of an off-the-shelf database container. In this case, the container for the database is supplied by the database project, and we would rather not modify that container simply to add health checks. Of course, a container orchestrator will allow us to add simple health checks to

ensure that the process is running and that it is listening on a particular port, but what if we want to add richer health checks that actually run queries against the database?

Container orchestration systems like Kubernetes enable us to use shell scripts as health checks as well. Given this capability, we can write a rich shell script that runs a number of different diagnostic queries against the database to determine its health. But where can we store such a script and how can we version it?

The answer to these problems should be easy to guess by now: we can use an adapter container. The database runs in the application container and shares a network interface with the adapter container. The adapter container is a simple container that only contains the shell script for determining the health of the database. This script can then be set up as the health check for the database container and can perform whatever rich health checks our application requires. If these checks ever fail, the database will be automatically restarted.

Hands On: Adding Rich Health Monitoring for MySQL

Suppose, then, that you want to add deep monitoring on a MySQL database where you actually run a query that is representative of your workload. In this case, one option would be to update the MySQL container to contain a health check that is specific to your application. However, this is generally an unattractive idea because it requires that you both modify some existing MySQL base image as well as update that image as new MySQL images are released.

Using the adapter pattern is a much more attractive approach to adding health checks to your database container. Instead of modifying the existing MySQL container, you can add an additional adapter container to the preexisting MySQL container, which runs the appropriate query to test the database health. Given that this adapter container implements the expected HTTP health check, it is simply a case of defining the MySQL database process's health check in terms of the interface exposed by this database adapter.

The source code for this adapter is relatively straightforward and looks like this in Go (though clearly other language implementations are possible as well):

```
package main

import (
    "database/sql"
    "flag"
    "fmt"
```

```
        "net/http"

        _ "github.com/go-sql-driver/mysql"
)

var (
        user   = flag.String("user", "", "The database user name")
        passwd = flag.String("password", "", "The database password")
        db     = flag.String("database", "", "The database to connect to")
        query  = flag.String("query", "", "The test query")
        addr   = flag.String("address", "localhost:8080",
                        "The address to listen on")
)

// Basic usage:
//   db-check --query="SELECT * from my-cool-table" \
//            --user=bdburns \
//            --passwd="you wish"
//
func main() {
        flag.Parse()
        db, err := sql.Open("localhost",
                        fmt.Sprintf("%s:%s@/%s", *user, *passwd, *db))
        if err != nil {
                fmt.Printf("Error opening database: %v", err)
        }

 // Simple web handler that runs the query
        http.HandleFunc("", func(res http.ResponseWriter, req *http.Request) {
                _, err := db.Exec(*query)
                if err != nil {
                        res.WriteHeader(http.StatusInternalServerError)
                        res.Write([]byte(err.Error()))
                        return
                }
                res.WriteHeader(http.StatusOK)
                res.Write([]byte("OK"))
                return
        })
 // Startup the server
        http.ListenAndServe(*addr, nil)
}
```

We can then build this into a container image and pull it into a pod that looks like:

```
apiVersion: v1
kind: Pod
metadata:
  name: adapter-example-health
  namespace: default
spec:
  containers:
  - image: mysql
    name: mysql
  - image: brendanburns/mysql-adapter
    name: adapter
```

That way, the `mysql` container is unchanged, but the desired feedback about the health of the MySQL server can still be obtained from the adapter container.

When looking at this application of the adapter pattern, it may seem like applying the pattern is superfluous. Clearly we could have built our own custom image that knew how to health check the `mysql` instance itself.

While this is true, this method ignores the strong benefits that derive from modularity. If every developer implements their own specific container with health checking built in, there are no opportunities for reuse or sharing.

In contrast, if we use patterns like the adapter to develop modular solutions comprised of multiple containers, the work is inherently decoupled and more easily shared. An adapter that is developed to health check `mysql` is a module that can be shared and reused by a variety of people. Further, people can apply the adapter pattern using this shared health-checking container, without having deep knowledge of how to health check a `mysql` database. Thus the modularity and adapter pattern serve not to just facilitate sharing, but also to empower people to take advantage of the knowledge of others.

Sometimes design patterns aren't just for the developers who apply them, but lead to the development of communities that can collaborate and share solutions between members of the community as well as the broader developer ecosystem.

Summary

Sometimes the world isn't perfectly aligned. The components that we need to use to build our systems can have different interfaces or different protocols for interactions like

logging, monitoring, and RPCs. Adapters can be a critical tool to add consistency to our systems, enabling them to share a consistent protocol and way of connecting. Using an adapter is often significantly easier than trying to change the upstream project.

Previous chapter

< 4. Ambassadors

Next chapter

III. Serving Patterns >